Notes on Deep RL

not even that “Deep” related


THE IMPORTANT THING ISN'T CAN
YOU READ MUSIC, IT'S CAN YOU HEAR
IT. CAN YOU HEAR THE MUSIC, ROBERT?

Where we left last time

So, up until now, we have seen some algorithms for discrete action space, and we then moved on some continuous action space algorithms:

- Deep Deterministic Policy Gradient, by exploiting the fact that the **Q** function is differentiable with respect to the action **a**
- Evolutionary strategies, which exploits random mutations in the parameter space, and moves towards the best solutions
- Actor critic methods, by parameterizing the policy as a Gaussian Distribution

Regarding this last one, we saw an example

One example is worth a thousand words

Let's step back from the word of DL for a second, and imagine we have the following problem:

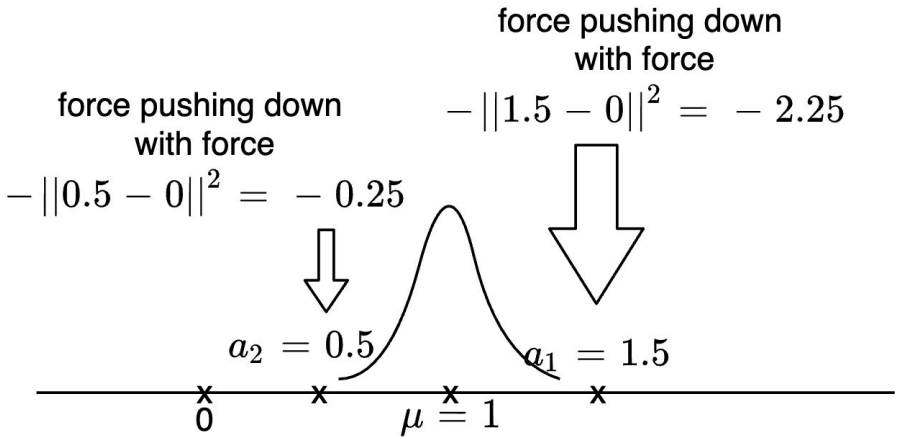
- the state is only one, so we don't need to model it
- the action is 1-dimensional and it is continuous in R
- the reward is the following $-||a - origin||^2$, thus the best action is the 0-action

Say we parameterize the policy with a Gaussian with fixed variance, thus we only want to learn μ

What we can do, is to sample from $\mathbf{N}(\mu, 1)$, and use RL to optimize μ

One example is worth a thousand words

To do so, say we initialize μ=1, and we sample 2 values from the distribution, **1.5 and 0.5**:



Given the 2 samples, we need to update μ using Gradient Ascent and REINFORCE:

$$\mu' = \mu + \alpha \nabla_\mu L$$

On the following loss:

$$\begin{aligned} \nabla_\mu L &= R \cdot \nabla_\mu \ln p(a|s) \\ &\approx R \cdot \ln e^{-(a-\mu)^2} \\ &= R \cdot -(a-\mu)^2 \end{aligned}$$

Have you ever seen anything similar to it?

Possibly in Deep/Machine Learning

Derivation of Negative Log Likelihood

Usually, the story goes like this:

1. you have some IID observations, and you want to model them with a probability distribution:

$$\arg\max_{\theta} p_\theta(x_1, x_2, \ldots, x_n)$$

$$\arg\max_{\theta} p_\theta(y_1, y_2, \ldots, y_n | x_1, x_2, \ldots, x_n)$$

Derivation of Negative Log Likelihood

Usually, the story goes like this:

1. you have some IID observations, and you want to model them with a probability distribution
2. you use the fact that they are independent (and some conditionally-independent assumptions that are not that important)

$$\arg\max_{\theta} \prod p_\theta(y_i|x_i)$$

Derivation of Negative Log Likelihood

Usually, the story goes like this:

1. you have some IID observations, and you want to model them with a probability distribution
2. you use the fact that they are independent
3. you exploit the fact that you are interested in the **max**, so you can do monotone transformations to it, and the max doesn't change

$$\begin{aligned}\arg\max_\theta \prod p_\theta(y_i|x_i) &= \arg\max_\theta \log\left(\prod p_\theta(y_i|x_i)\right) \\ &= \arg\max_\theta \sum \log p_\theta(y_i|x_i)\end{aligned}$$

Derivation of Negative Log Likelihood

Usually, the story goes like this:

1. you have some IID observations, and you want to model them with a probability distribution
2. you use the fact that they are independent
3. you exploit the fact that you are interested in the **argmax**, so you can do monotone transformations to it, and the argmax doesn't change
4. you assume a family of distribution for the conditional distribution, and you derive the log-probability

Classification in machine learning

For classification in Machine/Deep learning, usually one of 2 losses are used:

- Binary Cross Entropy (BCE), for the binary case:

$$\mathrm{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \cdot \log(f_\theta(x_i)) + (1 - y_i) \cdot \log(1 - f_\theta(x_i))\right]$$

a label that is 0 for the first class, 1 for the other one

output of the model, it being a logistic regression or a neural network

- Categorical Cross Entropy (CCE), for the multiclass case:

$$\mathrm{CCE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C} y_{ij} \cdot \log(f_\theta(x_i)_j)$$

a label that is 1 for the correct class, 0 for the others

Classification in machine learning

Now, let's not consider the mini-batch summation, and instead let's focus on the single sample:

$$\mathrm{CCE} = -\sum_{j=1}^{C} y_j \cdot \log(f_\theta(x)_j)$$

Does this look familiar to you?
Have you ever seen anything similar?

Classification in machine learning

Maybe considering a single class...

$$L(\theta) = -y \cdot \log(f_\theta(x))$$

Maybe removing the "-", since **y** is just a constant...

$$L(\theta) = C \cdot \log(f_\theta(x))$$

Or maybe considering the gradient with respect the model...

$$\nabla L(\theta) = C \cdot \nabla_\theta \log(f_\theta(x))$$

$$\mathrm{CCE} = -\sum_{j=1}^{C} y_j \cdot \log(f_\theta(x)_j)$$

this term is 1 for the correct answer, 0 otherwise

...almost like a reward!

...

...

$$\nabla L(\theta) = C \cdot \nabla_\theta \log(f_\theta(x))$$

...

...

and indeed it is, with an episode with only one step!

$$\nabla L(\theta) \approx \sum_{i=0}^{T} r_i \nabla \ln p_\theta(a|s) = G_t \nabla \ln p_\theta(a|s)$$


Always has been
wait, it was classification this whole time?
$\nabla L(\theta) \approx G_t \nabla \ln p_\theta(a|s)$

Reinforcement Learning as a classification

Let's put the two formulas side by side:

$$G_t \nabla \ln p_\theta(a|s) \qquad - y \nabla log(f_\theta(x))$$

Now, if we try to reformulate the REINFORCE formula in the "classification" one, what is doing, it's pretty much saying

*for the state **s** (image), the action **a** (label of the image) is the correct one, and I'm sure about it with "belief force" **G***

But... if discrete RL is almost just classification, what is continuous RL?

REINFORCE in continuous action space

Well, REINFORCE says pretty much just the following:

$$\nabla L(\theta) \approx \sum_{i=0}^{T} r_i \nabla \ln p_\theta(a|s)$$

It says nothing about the type of distribution we can use as **p(a|s)**, and since we are in a continuous setting, why not using the most used continuous distribution, the Gaussian distribution.

$$
\begin{aligned}
\nabla_\theta L(\theta) &\approx \sum_{i=t}^{T} r_i \nabla_\theta \ln p(a|s) \\
&= G_t \nabla_\theta \ln p(a|s) \\
&= G_t \nabla_\theta \ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{a-\mu_\theta}{\sigma}\right)^2} \right) \\
&= G_t \nabla_\theta \left[\ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) + \ln \left(e^{-\frac{1}{2}\left(\frac{a-\mu_\theta}{\sigma}\right)^2} \right) \right] \\
&= G_t \nabla_\theta \ln \left(e^{-\frac{1}{2}\left(\frac{a-\mu_\theta}{\sigma}\right)^2} \right) \\
&= G_t \nabla_\theta - \frac{1}{2} \left(\frac{a-\mu_\theta}{\sigma} \right)^2 \\
&= \frac{-G_t}{2\sigma^2} \nabla_\theta \left(a - \mu_\theta \right)^2
\end{aligned}
$$

REINFORCE

explicitly writing the PDF

$$\ln(a \cdot b) = \ln(a) + \ln(b)$$

removing the constant form the gradient

simplifying the exponential and the logarithm

moving out the factor from the gradient

REINFORCE in continuous action space

Well, if the gradient of the loss is: $\frac{-G_t}{2\sigma^2}\nabla_\theta (a - \mu_\theta)^2$

$$\theta' = \theta - \alpha\frac{-G_t}{2\sigma^2}\nabla_\theta (a - \mu_\theta)^2$$

We can do Gradient descent on it:

$$= \theta - \frac{\alpha}{2\sigma^2}G_t\nabla_\theta (\mu_\theta - a)^2$$

$$= \theta - \beta G_t\nabla_\theta (\mu_\theta - a)^2$$

REINFORCE in continuous action space

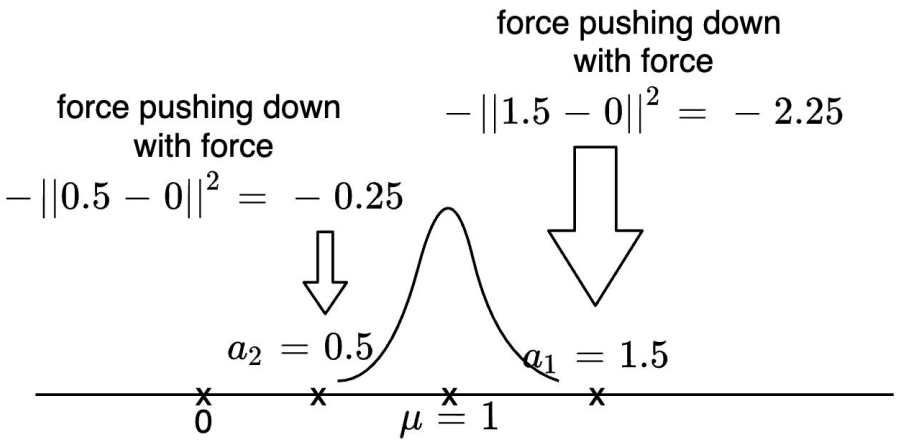
As per the discrete case, let's compare the REINFORCE loss with the regression loss:

$$\nabla_\theta (y_\theta - t)^2 \qquad G_t \nabla_\theta (\mu(s)_\theta - a)^2$$

Now, if we try to reformulate the REINFORCE formula in the "regression" one, what is doing, it's pretty much saying

*for the state **s** (image), the action **a** (target of the regression) is the correct one, and I'm sure about it with "belief force" **G***

One example is worth a thousand words



Given the 2 samples, we need to update $\boldsymbol{\mu}$ using Gradient Ascent and REINFORCE:

$$\mu' = \mu + \alpha \nabla_\mu L$$

On the following loss:

$$\begin{aligned}\nabla_\mu L &= R \cdot \nabla_\mu \ln p(a|s) \\ &\approx R \cdot \ln e^{-(a-\mu)^2} \\ &= R \cdot -(a-\mu)^2\end{aligned}$$